

2026年实验班大一寒假大作业：全栈AIoT智能音箱系统开发

一、任务背景与目标

在人工智能与物联网（AIoT）深度融合的时代，掌握从底层硬件驱动到云端大模型集成的全栈开发能力是实验班学生的必备素质。本次大作业要求各小组基于Linux、Python/Micropython 和单片机的课程内容，结合一本即将出版的教材《从零构建大语言模型与智能硬件》，亲手打造一个具备**语音唤醒**、**连续对话**、**实时搜索能力**的智能音箱系统，并在此基础上进行跨设备的创新应用开发。

请所有同学下载并双面打印本教材草稿：

【金山文档 | WPS云文档】 AIoT 教材草稿-20260120

<https://365.kdocs.cn/l/cvpW4SsSy4sj>

请所有同学用Gitee 下载教材配套代码到本地文件夹内：

https://gitee.com/turcar_0/AIoT-textbook-code.git

注意：本次大作业的架构是一个“双端一云”的分布式 **AIoT** 架构，具体分为以下三个层级：

1. 云端中枢层（The Cloud Brain）

- 核心服务**：基于 **FastAPI** 的异步 Web 服务器。它不再仅仅是一个简单的中转站，而是一个**状态机管理器**。
- 数据存储**：利用 **Redis** 存储多用户的对话上下文（Memory）和设备状态（如：当前音箱是处于“聆听”还是“播放”状态）。
- AI 引擎**：集成火山引擎豆包大模型（逻辑推理）、百度 ASR（耳朵）和百度 TTS（嘴巴）。
- 通信协议**：同时支持 **WebSocket**（用于音箱端的实时音频流）和 **HTTP RESTful**（用于紫色板的状态查询与控制）。

2. 边缘感知与交互层（The Edge Nodes）

- 节点 A（智能音箱）**：基于 ESP32-S3。负责最沉重的任务——本地语音唤醒（WakeNet）、静音检测（VAD）以及音频的原始二进制流传输。它是系统的“主交互界面”。
- 节点 B（创意控制板）**：基于 ESP32-D0WD。负责环境感知与物理辅助。它通过 MicroPython 运行轻量级逻辑，是系统的“感知外挂”和“视觉反馈器”。

3. 数据流转逻辑（Data Flow）

- 上行流**：音箱捕捉语音 -> 转化为二进制流 -> WebSocket 传至云端 -> ASR 转文字 -> LLM 生成回复。
- 下行流**：LLM 回复 -> TTS 转音频 -> WebSocket 传回音箱播放；同时，回复中的指令信息 -> 存入 Redis -> 紫色板通过 HTTP 获知指令 -> 触发 LED/蜂鸣器动作。

本次大作业架构与教材架构的区别与联系

1. 类似之处：

- 核心链路一致**：完全保留了教材中 ASR -> LLM -> TTS 的处理逻辑。
- 技术栈一致**：依然使用教材推荐的 FastAPI、Redis、Docker 和 ESP-IDF。

2. 主要区别（针对实际能力的优化）：

- 从“单兵”到“协同”：教材架构是“1个音箱 + 1个后端”。大作业架构是“1个后端 + 1个音箱 + 1个传感器节点”。这要求学生必须处理多设备间的同步问题（例如：如何确保音箱说话时，紫色板的灯光能准时亮起）。
- 去繁就简：去掉了教材后期复杂的 RabbitMQ 消息队列。在 大一阶段，直接利用 FastAPI 的异步特性和 Redis 的发布/订阅功能足以应对 2-3 个节点的并发，降低了运维难度。
- 开发语言多元化：教材是纯 C++ 与 Python。大作业引入了 KIMI, TRAE 和 MicroPython，降低了创意组的硬件开发门槛，让他们能把精力放在“场景设计”而非“内存管理”上。

总之：
本次作业的架构并非教材中简单的“1对1”结构，而是一个“双端一云”的分布式架构：

- 云端大脑 (Cloud Hub)：部署在组长的学生云主机上。运行 FastAPI 异步服务器，集成豆包大模型、百度语音服务，并利用 Redis 维护全局对话状态。
- 交互终端 (Speaker Node)：基于 ESP32-S3。负责高强度的实时音频流处理（C++/ESP-IDF），是用户与 AI 对话的物理入口。
- 感知外挂 (Creative Node)：基于 ESP32-D0WD（紫色板）。负责环境感知与物理反馈（MicroPython），通过 HTTP 协议与云端大脑同步，实现视觉或触觉的辅助交互。

与教材的区别：教材侧重于“如何做一个音箱”，而大作业侧重于“如何构建一个多设备协同的智能空间”。你们需要处理多端同步逻辑，这更接近真实的工业物联网场景

如何确保能学会

- 教材先行：全组同学必须在第一周（2月7日前）把教材 1-10 章看完，把本地 Ubuntu 的 FastAPI 跑通，并整理成一份简单的 API 接口文档（例如：访问哪个 URL 可以拿到最后一句对话）。
- 音箱组攻坚：音箱组不要急着写代码，先用教材提供的 test_client.py 在电脑上模拟音箱，确认能听到“韩立老师”说话后，再往硬件上搬迁代码。
- 创意组联动：创意组在第二周必须加入联调，你们的任务是“蹭”组长的接口。只要组长的服务器在线，你们的紫色板就能通过简单的 json() 解析获得 AI 的智慧。

二、硬件准备

每组需准备以下硬件：

- 云端服务器：由组长负责，在腾讯云或阿里云购买“学生优惠版”轻量应用服务器（最低配置即可，需安装 Ubuntu 22.04 LTS，只需要购买 1 个月即可）。
- 音箱开发套件（2套）：由音箱组负责购买。
 - 购买链接：<https://e.tb.cn/h.7lvQ3INPythiWTO?tk=PONCUimFuPF>
 - 核心组件：ESP32-S3 开发板、INMP441 数字麦克风、MAX98357 功放模块、小喇叭。
- 紫色开发板套件（学校已经发放）：由创新组负责。
 - 核心组件：ESP32-D0WD 核心板、面包板、LED灯、蜂鸣器、4个独立按钮、杜邦线。

三、核心学习策略：AI 辅助与双系统开发

考虑到大家目前主要掌握 Python 和 MicroPython，而教材涉及硬核的 C++（ESP-IDF 框架）和复杂的后端架构，特制定以下策略：

- 环境基石：所有同学必须在个人电脑上安装 Windows + Ubuntu 双系统（非教材中说的虚拟机）。原生 Linux 环境能确保 ESP32 串口挂载的稳定性。
- 大模型编程工具Kimi Code CLI 或TRAE 辅助：

- 快速跑通：在本地电脑克隆教材代码后，使用在kimi.com 网站下载 `kimi-code` 或trae.cn 网站下载TRAE IDE 扫描目录，自动补全缺失的依赖环境。
- 代码翻译：遇到教材中深奥的 C++ 指针或 FreeRTOS 任务调度逻辑时，使用 Kimi Code 进行“代码逐行解释”，将其逻辑与你们熟悉的 MicroPython 进行类比学习。

3. 精简章节：本次作业舍弃 教材中Ch 21 (RabbitMQ) 与 Ch 22 (Locust) ，重点攻克 WebSocket 实时通信与 VAD 语音检测。

四、团队分工与具体要求

1. 组长（1名）：云端架构师

- 负责章节：Ch 1-12, Ch 23。
- 具体任务：
 - 第一阶段：在本地 Ubuntu 环境下利用 Docker 部署 Redis 和 FastAPI 后端，确保 API 能正常响应。
 - 第二阶段：配置云服务器安全组，将后端服务迁移至云端，获取公网 IP。
 - 技术要求：必须掌握 `docker-compose` 一键启动脚本，确保后端服务在音箱组调试时不间断运行。

2. 音箱开发组（2名）：嵌入式工程师

- 负责章节：Ch 13-20。
- 具体任务：
 - 硬件集成：按照 Ch 14 完成麦克风与功放的电路连接。
 - 固件开发：使用 C++/ESP-IDF 编写固件。重点实现 Ch 18 的 **WakeNet（语音唤醒）** 和 Ch 19 的 **VAD（静音检测）**。
 - 联调：将音箱通过 WebSocket 连接到组长的服务器，实现“听到唤醒词 -> 录音 -> 云端解析 -> 播放回复”的闭环。

3. 创新应用组（2名）：交互设计师

- 负责任务：自主设计。
- 具体任务：
 - 开发语言：使用 **MicroPython**。
 - 联动逻辑：紫色开发板作为音箱的“外挂感知器”或“物理控制器”。
 - 启发方向：
 - 场景A（视觉联动）：音箱在思考时，紫色板的 LED 阵列显示流光灯效；音箱说话时，LED 随音量闪烁。
 - 场景B（物理控制）：紫色板上的 4 个按钮分别对应“翻译模式”、“音乐模式”、“紧急求助”和“静音”。按下按钮，通过组长的云服务器向音箱推送指令。
 - 场景C（环境预警）：利用紫色板模拟传感器，当检测到特定信号（如按钮模拟火警），强制中断音箱当前对话，发出蜂鸣报警并语音播报。

1. 为什么要单独设立“创意”环节？

- 从“孤立设备”到“分布式系统”：教材教的是如何做一个“智能音箱”，而创意组的任务是将音箱变成“智能空间”的一部分。这培养了学生系统级架构的思维——即 AI 不仅仅存在于音箱里，还可以通过传感器和执行器延伸到物理世界的每一个角落。

- **多模态交互的补完**：音箱是“听觉”交互，紫色板通过 LED、蜂鸣器和按钮提供了“视觉”和“触觉”交互。这种多模态的联动才是现代 AloT 产品的核心竞争力。

2. 与教材的结合与拓展点：

- **结合点**：创意组必须调用组长在教材中实现的 **FastAPI 接口**。这意味着他们需要理解教材中关于 RESTful API、JSON 数据格式和 HTTP 协议的全部内容。
- **拓展点**：教材中音箱是“被动唤醒”的。创意组可以想象一下如何实现“主动触发”。

4. 注意：

不管如何分工，组内所有人必须学习教材 1-23 章！所有人必须在本地电脑里完成 教材 1-20 章的调试。每组可以把教材上传到一个大模型（比如Kimi 2.5），把学校发的紫色开发板及所有配件、音箱开发套件及配件、本次大作业具体要求、每组同学地理位置等详细信息都提交给大模型，学会与大模型多轮对话，共同找到“创意”的场景。

学院报销：

1. 云主机费用（1 个月且务必是最便宜的学生云主机）
 2. 两套音箱开发套件的费用
- 我们不报销 KIMI Code的token费，大家可以尽量使用赠送的套餐。你也可以使用其他厂家的类似产品：TRAE IDE 或腾讯云CodeBuddy 等。
- 有能力的同学可以使用课上讲过的Claude Code+GLM4.7 或+MiniMax或+Kimi 2.5 等国产开源大模型。

五、 时间节点与作业提交（QQ实验班公告群视频）

阶段一：个体筑基与本地实验（2月1日 - 2月8日）

学习范围：教材 Ch 1 - Ch 10（后端逻辑与 Python 基础）+ Ch 13 - Ch 14（硬件基础与回声实验）。

- **要求**：每位同学必须在本地电脑（双系统 Ubuntu）跑通教材前 10 章的代码。
- **汇报视频（2月8日提交）**：5 人分别录制。
 - **组长**：展示本地 Docker 启动 Redis 和 FastAPI，通过 `curl` 成功调用豆包 API。
 - **音箱组（2人）**：展示 ESP32-S3 硬件连接，跑通 Ch 14 的离线“回声实验”。
 - **创意组（2人）**：展示紫色板跑通 MicroPython，实现 Wi-Fi 连接并能通过本地电脑串口代码控制 紫色板LED 和蜂鸣器。
- **工具建议**：使用 **Kimi Code CLI** 或 TRAE IDE解释教材中复杂的 C++ 结构体定义。

阶段二：云端集成与音箱上线（2月9日 - 2月16日）

学习范围：教材 Ch 11 - Ch 12（WebSocket）+ Ch 17 - Ch 20（语音唤醒与 VAD）+ Ch 23（Docker 部署）。

- **核心任务**：音箱脱离本地电脑，直接连接公网云主机。
- **汇报视频（2月16日/腊月廿九提交）**：小组集体成果。
 - 展示组长云主机上的 Docker 容器正常运行。
 - 展示音箱组的 ESP32-S3 喊出“你好小智”后，能通过 Wi-Fi 访问云端并流畅回答问题。
- **注意**：2月17日-19日（除夕至初二）为休息时间，不安排任务。

阶段三：创意联动与教材审计（2月20日 - 2月25日）

学习范围：全书复盘 + 自主设计。

- **核心任务**：完成紫色板与音箱的联动，并对教材进行深度纠错。
- **汇报视频（2月25日提交）**：
 - 展示“创意联动”场景。例如：某同学按下本地紫色板按钮，远程的另外一个同学的音箱主动播报天气；或远程的另外一个音箱说话时，本地紫色板 LED 随说话而跳动。
- **文档提交（2月25日群内 PDF提交）**：
 - 提交一份《教材纠错与改进建议》PDF。**必须覆盖每一章**，指出逻辑漏洞（如：某处变量未定义）、代码 Bug（如：C++ 语法错误）或环境坑点（如：某库版本已更新导致不兼容）。

六、老师寄语

同学们，大一的学习不应止于纸笔。这本教材是你们进入后续比赛和科研项目的“入场券”。C++ 很难，云部署很杂，但有了 Kimi Code 和 TRAE的辅助和小组的协作，你们一定能攻克。考虑到大家寒假期间分布在全国各地，协同开发确实存在物理距离上的挑战。为了确保每位同学都能扎实掌握底层原理，同时又能高效完成小组集成任务，请务必协作起来，用大模型和集体的智慧来完成本次大作业。

注意：除夕至大年初二（2月17日-19日）为休息时间，请大家陪家人过个好年！